

2.1.1 Estrategias y Enfoques de Diseño Plan de Pruebas con Calidad y Seguridad

Guía completa para crear planes de pruebas robustos que garanticen la



Contenido

01

I. Introducción y Fundamentos

02

02. Simplicidad y Componentes Esenciales del Plan

05

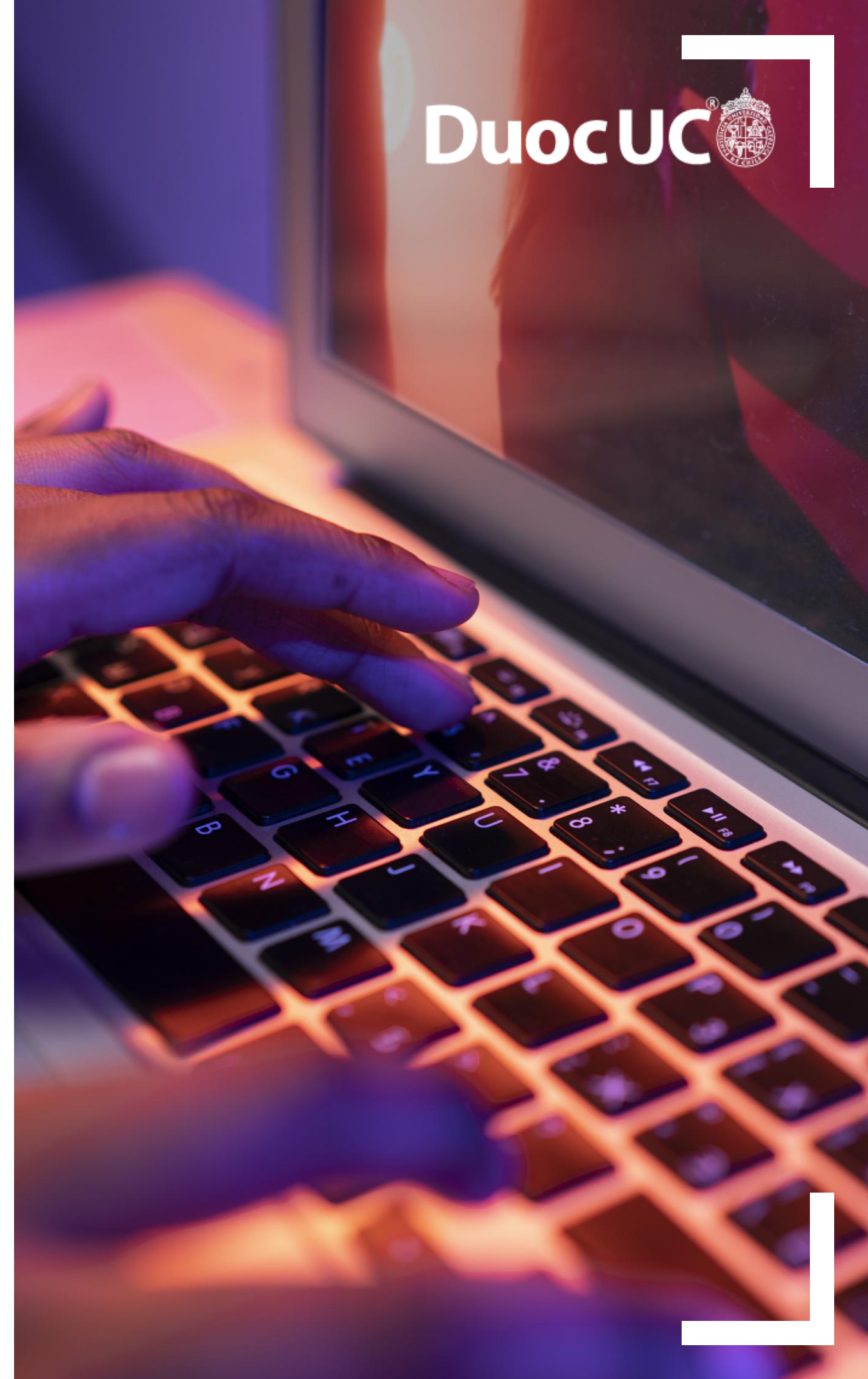
05. Documentación, Comunicación y Cierre

03

03. Estrategia, Planificación y Herramientas

04

04. Plan de Pruebas en Entornos Modernos (Docker)



Estrategias y Enfoques de Diseño del Plan de Pruebas con Calidad y Seguridad.

Guía completa para crear planes de pruebas robustos que garanticen la calidad

```
.App-logo {  
  height: 40vmin;  
  pointer-events: none;  
}  
  
@media (prefers-reduced-motion: no-preference) {  
  .App-logo {  
    animation: App-logo-spin infinite 20s linear;  
  }  
}  
  
.App-header {  
  background-color: #282c34;  
  min-height: 100vh;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  font-size: calc(10px + 2vmin);  
}  
  
.App-link {  
  color: #61dafb;  
}  
  
@keyframes App-logo-spin {  
  from {  
    transform: rotate(0deg);  
  }  
  to {  
    transform: rotate(360deg);  
  }  
}
```

El "Check-In" de Calidad

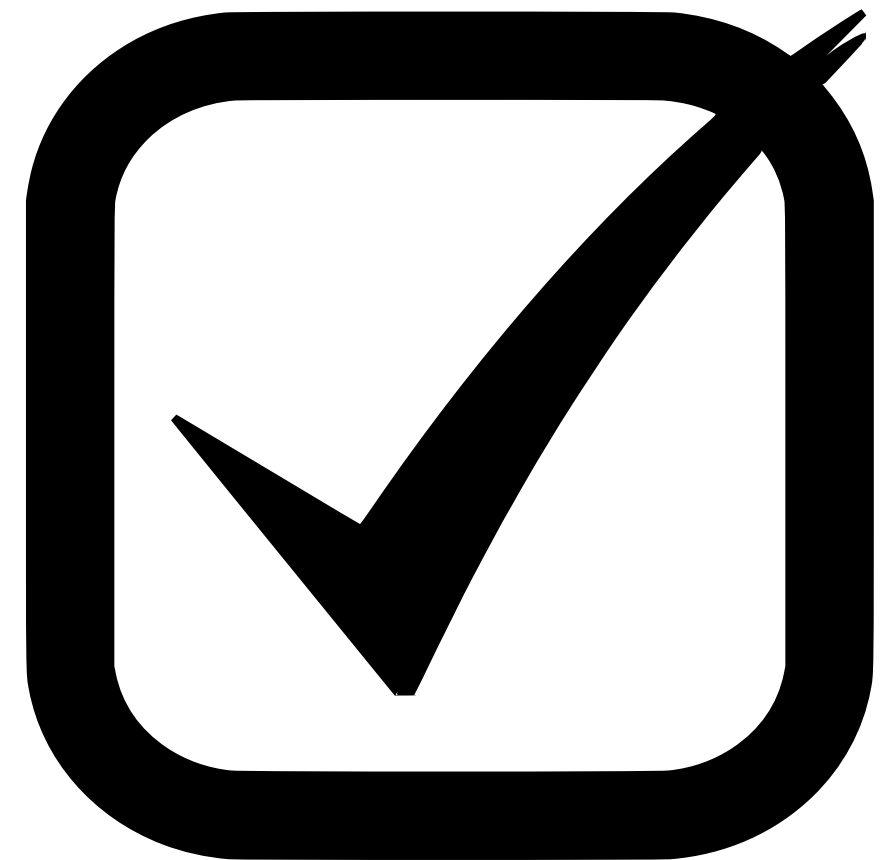
Objetivo: Conectar la experiencia personal con el propósito fundamental de un plan de pruebas.

Dinámica: "¿Qué tan robusto es tu 'Escudo'?"

Imaginen que están a punto de lanzar una aplicación que manejará los ahorros de toda su vida. Antes de presionar "Publicar", respondan rápidamente:

- **El Punto Crítico:** Si solo pudieran probar **una sola cosa** para dormir tranquilos hoy, ¿probarían que el botón de "Enviar" funcione (Calidad) o que nadie les robe la contraseña (Seguridad)?.
- **El Riesgo Fantasma:** ¿Cuál es el error más "caro" que han cometido o visto en un desarrollo por no probar a tiempo?.

Idea clave: "La calidad no es un accidente, es el resultado de una ejecución disciplinada". Un plan robusto es el marco estratégico que nos da esa disciplina.



1: Introducción al Plan de Pruebas con Enfoque en Calidad y Seguridad

Introducción al Plan de Pruebas

¿Qué es un plan de pruebas?

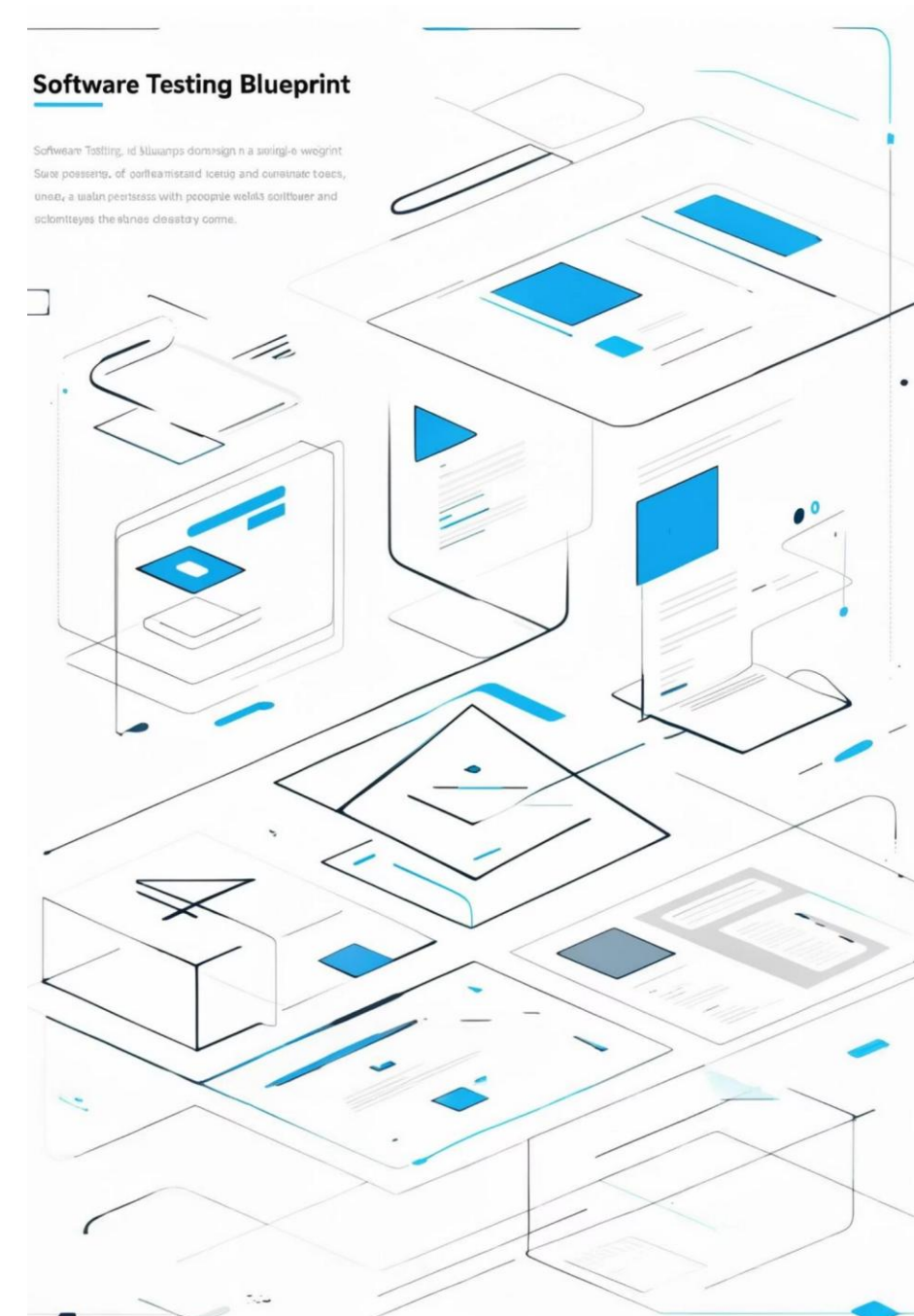
Un marco estratégico que organiza, estructura y guía todas las actividades de validación del producto de software.

Propósito fundamental

Asegurar la calidad del producto, detectar defectos en etapas tempranas y validar la seguridad antes del lanzamiento.

Importancia crítica

Reduce riesgos significativamente, mejora la comunicación entre equipos y garantiza entregas confiables al cliente.



¿Por qué integrar calidad y seguridad en el plan de pruebas?

Enfoque en Calidad

Asegura el cumplimiento riguroso de los requisitos funcionales y no funcionales, garantizando que el software responda eficazmente a las expectativas del usuario.

- Validación exhaustiva de la funcionalidad
- Rendimiento eficiente y sostenido
- Experiencia de uso intuitiva
- Amplia compatibilidad con entornos y plataformas

Enfoque en Seguridad

Implementa medidas proactivas para prevenir vulnerabilidades, mitigar amenazas y proteger la integridad de los datos y sistemas críticos.

- Prevención de brechas y accesos no autorizados
- Protección robusta de la información sensible
- Alineación con normativas y estándares vigentes
- Fortalecimiento de la confianza del usuario

La integración de ambos enfoques permite desarrollar productos robustos, confiables y seguros, alineados con las exigencias del entorno digital actual.



2: Elementos Clave de un Plan de Pruebas Efectivo

01

Alcance y objetivos claros

Definición precisa de qué se prueba, qué no se incluye y por qué.

03

Recursos y roles

Equipos definidos con responsabilidades específicas y cronograma realista.

02

Estrategia de pruebas

Tipos, niveles y enfoques: manual, automatizado, basado en riesgos.

04

Criterios de aceptación

Condiciones de entrada y salida, identificación de riesgos y estrategias de mitigación.

Alcance y Objetivos

Definición de funcionalidades críticas

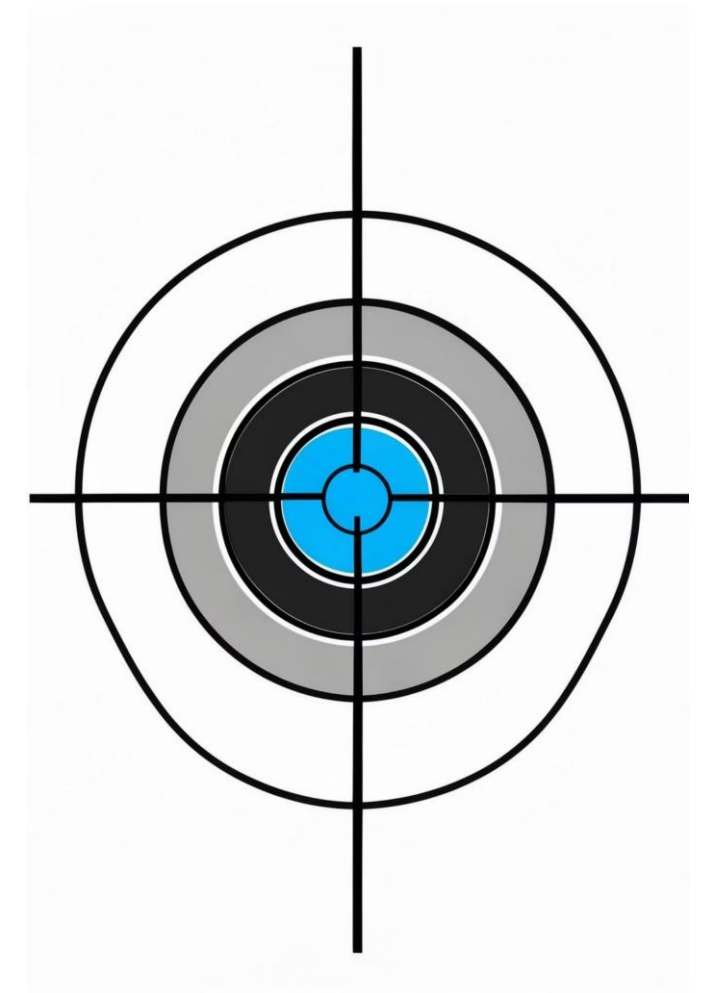
Identificar las áreas de mayor impacto en el negocio y las funcionalidades que requieren mayor atención en las pruebas.

Áreas de seguridad prioritarias

Mapear los módulos expuestos a amenazas potenciales y establecer controles de seguridad necesarios.

Objetivos específicos medibles

Validar funcionalidades core, verificar rendimiento bajo carga y asegurar protección de datos sensibles.



 **Ejemplo práctico:** "Verificar que el sistema de autenticación implementa cifrado seguro, maneja correctamente las sesiones y previene accesos no autorizados."

Estrategia y Enfoque de Pruebas

Pruebas Funcionales

Validación de requisitos del negocio y flujos de usuario principales.

Pruebas de Integración

Verificación de comunicación entre módulos y sistemas externos.

Pruebas de Rendimiento

Evaluación de velocidad, escalabilidad y estabilidad bajo carga.

Enfoque basado en riesgos

Priorizar las pruebas en módulos con mayor impacto potencial en el negocio y los usuarios.

Automatización estratégica

Incorporar pruebas automatizadas para regresión continua y validación de seguridad recurrente.

3: Diseño de Casos de Prueba con Enfoque en Calidad y Seguridad

Casos funcionales

Validar requisitos del negocio y flujos completos de usuario

Casos de seguridad

Autenticación, autorización y validación de entradas críticas

Guías OWASP

Aplicación de estándares para pruebas de seguridad web



Los casos de prueba deben ser específicos, medibles, reproducibles y trazables a los requisitos originales. Cada caso debe incluir precondiciones claras, pasos detallados y resultados esperados verificables.

Principios Fundamentales del Testing Aplicados



Pruebas Tempranas

Detectar defectos y vulnerabilidades desde las fases iniciales del desarrollo reduce costos exponencialmente.



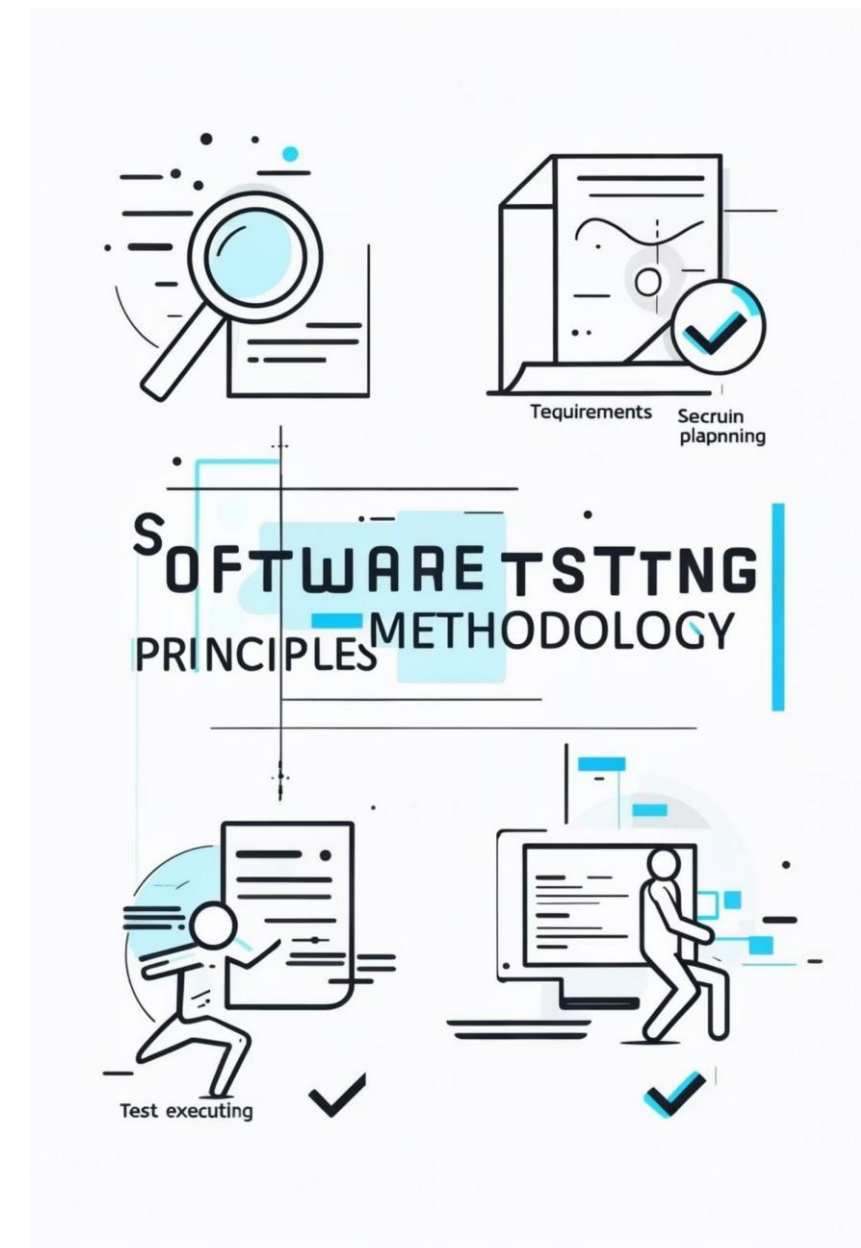
Testing Enfocado

El testing exhaustivo no es posible: priorizar áreas críticas y de alto riesgo es fundamental.



Demostración de Defectos

La ejecución de pruebas demuestra la presencia de defectos, pero no su ausencia total.



Ejemplo Práctico: Caso de Prueba de Seguridad

Validación contra Inyecciones SQL

01

Objetivo

Validar que el sistema rechaza y registra intentos de inyección SQL maliciosa.

03

Resultado esperado

El sistema bloquea el ataque, no expone información sensible y registra el evento en logs de seguridad.

02

Pasos de ejecución

Enviar entradas maliciosas en campos de formulario, verificar respuesta del sistema y analizar manejo de errores.



❏ Entrada maliciosa típica: ' OR '1'='1' --

4: Planificación de Recursos y Roles



Independencia del Testing

Equipos independientes de desarrollo y testing garantizan objetividad y perspectiva crítica en la evaluación.



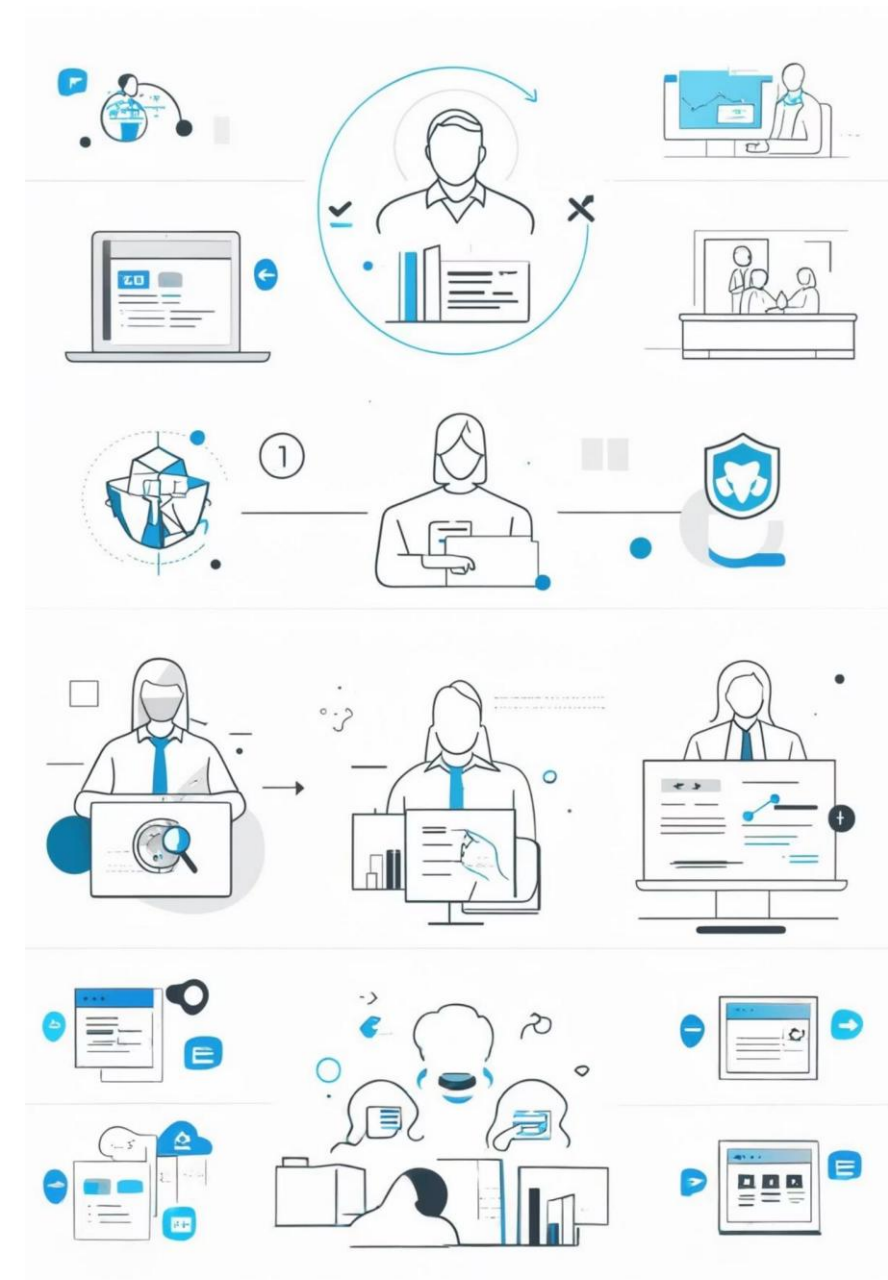
Roles Especializados

Líder de QA, testers funcionales, especialistas en automatización y expertos en seguridad trabajando coordinadamente.

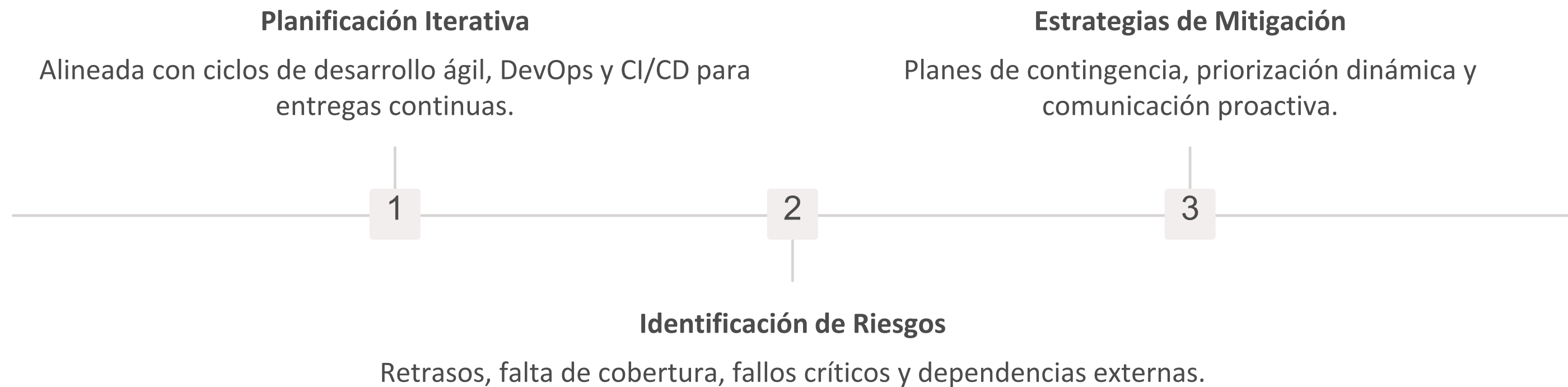


Herramientas Adecuadas

Entornos de prueba aislados, frameworks de automatización y escáneres de vulnerabilidades actualizados.



Cronograma y Gestión de Riesgos



Ejemplo de priorización

Ejecutar primero las pruebas de seguridad en módulos expuestos directamente a internet, como APIs públicas, formularios de autenticación y pasarelas de pago.

5: Integración de Automatización y Herramientas

1

Automatización de Regresión

Pruebas repetitivas ejecutadas automáticamente en cada cambio de código para detectar regresiones temprano.

2

Seguridad Continua

Análisis estático (SAST) y dinámico (DAST) integrados en el pipeline de desarrollo.

3

Herramientas Especializadas

Selenium para pruebas funcionales, OWASP ZAP y Burp Suite para seguridad web.

70%

Reducción en tiempo

La automatización reduce el tiempo de ejecución de pruebas de regresión.

3x

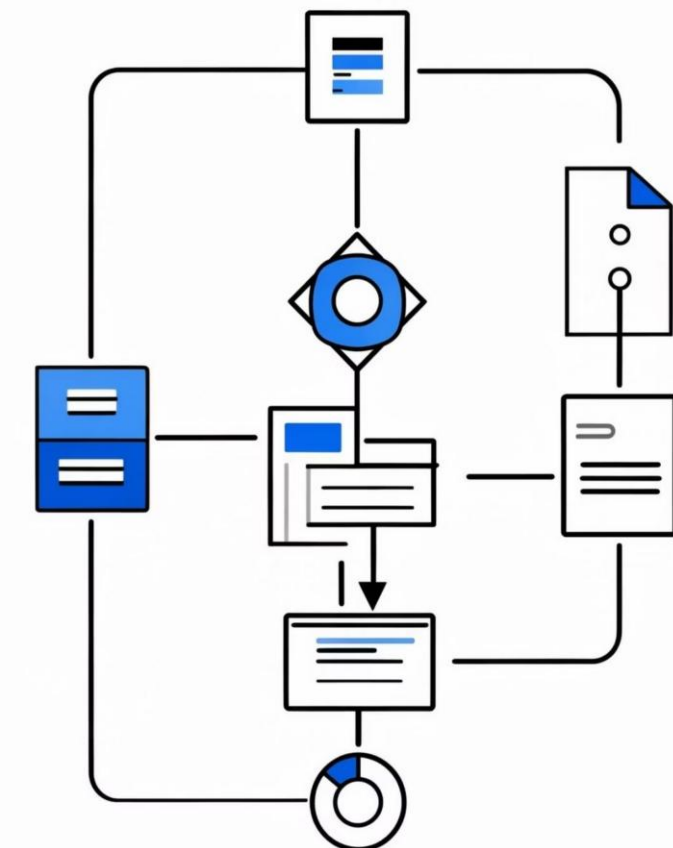
Mayor cobertura

Se puede ejecutar tres veces más casos de prueba con la misma inversión.

95%

Consistencia

Las pruebas automatizadas mantienen consistencia en cada ejecución.



Evaluación y Métricas de Calidad

Indicadores de Calidad

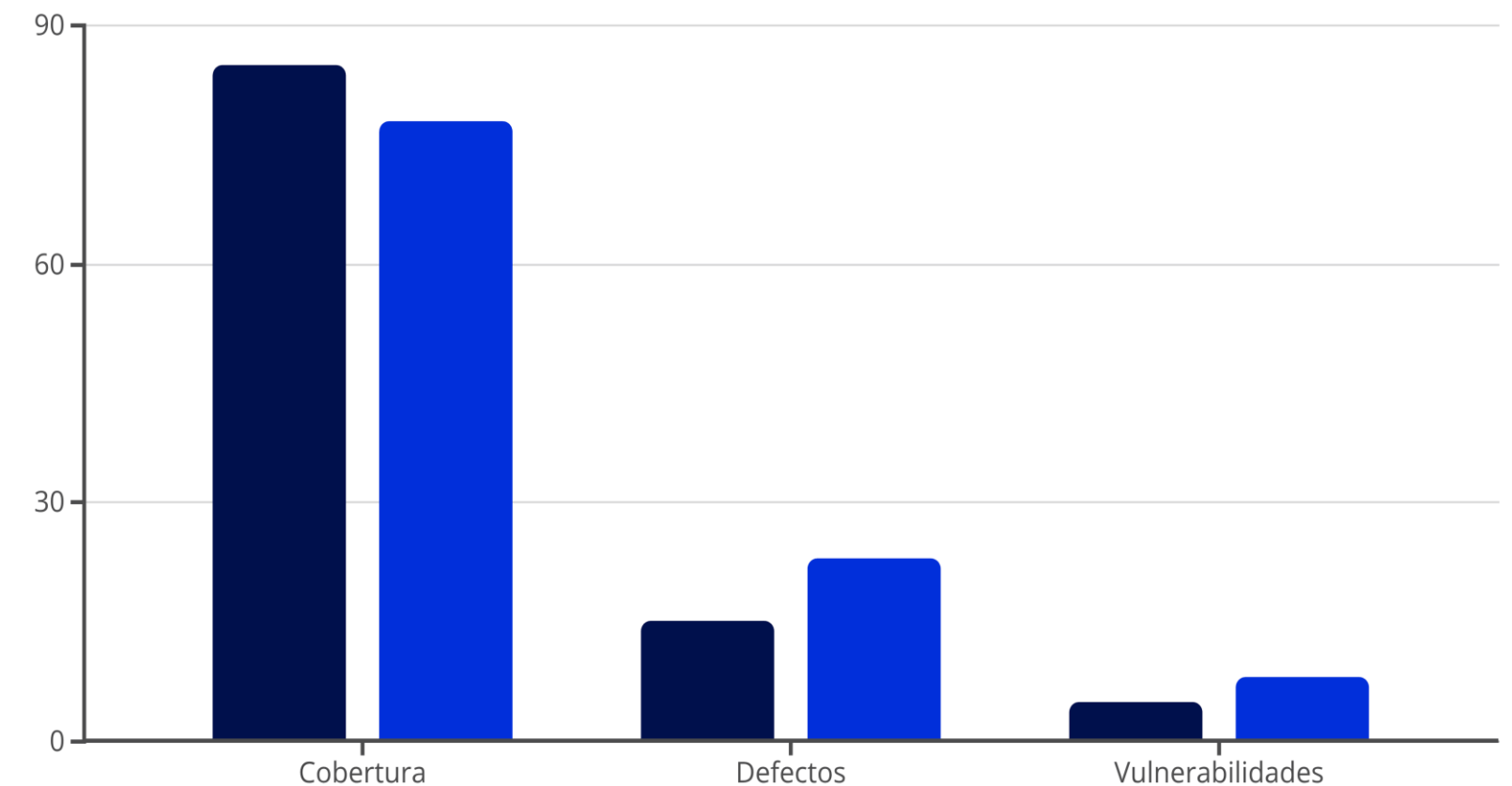
Miden la eficacia del proceso productivo y la conformidad del producto con los estándares establecidos.

- Tasa de defectos por millón de unidades
- Calificación del producto manufacturado según criterios técnicos
- Tiempo promedio requerido para inspección y control
- Eliminación de tolerancias críticas en factores operativos

Indicadores de Seguridad

Evalúan el nivel de protección, respuesta y cumplimiento en entornos operativos.

- Número total de incidentes de seguridad registrados
- Tiempo promedio de respuesta ante incidentes críticos
- Porcentaje de uso correcto de Equipos de Protección Personal (EPP)
- Calificación promedio del Sistema de Administración de Seguridad (SAS) en el Top 10 de áreas evaluadas



6: Documentación y Comunicación



La comunicación efectiva entre desarrollo, QA, seguridad y negocio es fundamental para el éxito del plan. Establezca canales claros y reuniones regulares de sincronización.

Caso Real: Implementación Exitosa de un Plan con Enfoque en Seguridad

Contexto: Empresa X

Una empresa de e-commerce que procesaba datos sensibles de clientes enfrentaba vulnerabilidades críticas antes del lanzamiento.

01

Integración OWASP WSTG

Incorporaron la guía completa de pruebas de seguridad web en su plan.

02

Detección Proactiva

Identificaron 47 vulnerabilidades, 12 de ellas críticas.

03

Corrección Temprana

Todas las vulnerabilidades fueron corregidas antes del lanzamiento.

70%

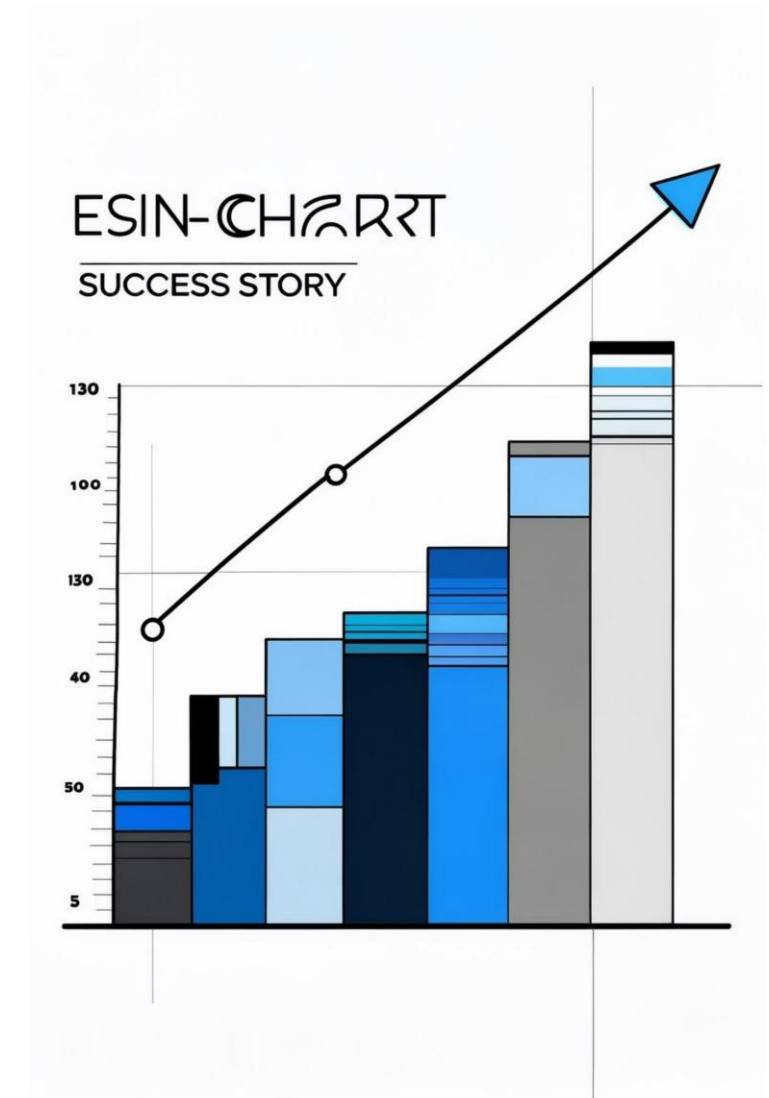
Reducción de incidentes

Menos incidentes de seguridad post-producción comparado con proyectos anteriores.

\$250K

Ahorro estimado

En costos de remediación y daños reputacionales evitados.



Desafíos Comunes y Cómo Superarlos

Pruebas Tardías

Las pruebas que inician tarde retrasan la detección de fallos críticos y aumentan costos de corrección.

Solución: Implementar Shift-Left Testing, integrando pruebas desde las primeras fases del diseño.

Falta de Recursos Especializados

Escasez de expertos en seguridad y pruebas avanzadas limita la efectividad del plan.

Solución: Invertir en capacitación continua, certificaciones y mentorías internas para desarrollar talento.

Desalineación entre Equipos

Falta de comunicación genera expectativas diferentes y pruebas incompletas.

Solución: Establecer ceremonias regulares, documentación compartida y objetivos comunes claros.

Herramientas y Recursos para Crear tu Plan de Pruebas

Plantillas y Frameworks

- Plantillas estructuradas IEEE 829
- Frameworks ágiles de testing
- Modelos de casos de prueba reutilizables
- Checklist de criterios de aceptación



Automatización

Selenium, Cypress, Playwright, Appium

Guías y Estándares

- OWASP Testing Guide (WSTG)
- OWASP Top 10 vulnerabilidades
- ISTQB Syllabus y mejores prácticas
- NIST Cybersecurity Framework



Seguridad

OWASP ZAP, Burp Suite, SonarQube



Gestión

Jira, TestRail, Azure DevOps, Zephyr



Estrategia de Pruebas: Flujo de Trabajo Optimizado

- 1. Definir alcance y objetivos medibles** Delimitar qué se evaluará, establecer prioridades y formular criterios de éxito verificables.
- 2. Diseñar una estrategia basada en riesgos** Seleccionar tipos de pruebas y enfoques adecuados para áreas críticas, considerando impacto y probabilidad.
- 3. Planificar recursos y cronograma operativo** Asignar roles, herramientas y tiempos realistas para cada fase del proceso de prueba.
- 4. Elaborar casos de prueba exhaustivos** Redactar casos funcionales y de seguridad con criterios claros de validación y trazabilidad.
- 5. Integrar automatización y métricas clave** Implementar herramientas automatizadas y definir indicadores (KPIs) para medir cobertura, eficacia y eficiencia.
- 6. Documentar y revisar de forma continua** Mantener registros actualizados, incorporar hallazgos y ajustar la estrategia conforme evoluciona el sistema.



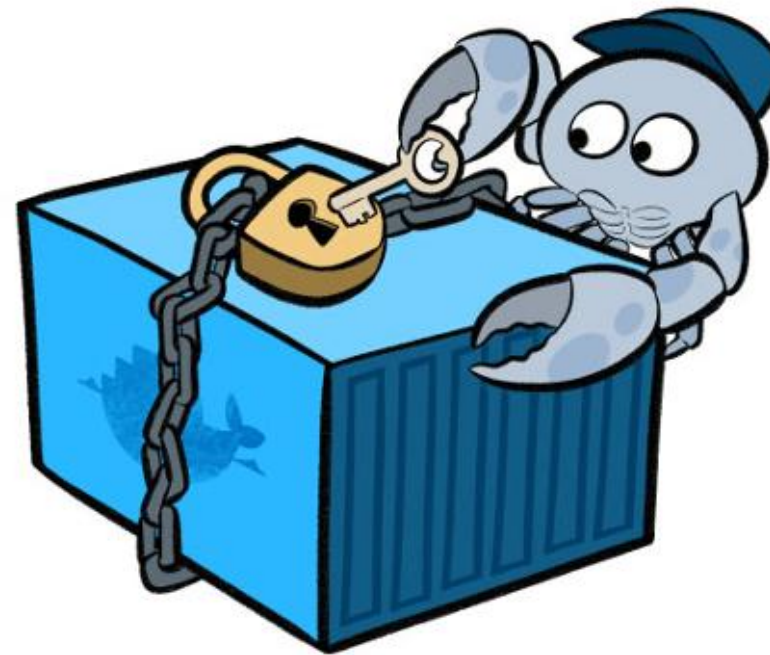
Pausa Activa: El Desafío de la Perspectiva

En lugar de ejercicios físicos, tómate un minuto para responder mentalmente la siguiente **pregunta de introspección técnica**:

¿Si hoy un atacante lograra saltarse tu sistema de autenticación, tu Plan de Pruebas actual tiene un caso diseñado para detectar cómo sucedió?

- **Si la respuesta es NO:** Es momento de revisar la integración de las **Guías OWASP** y el diseño de casos de seguridad en tu estrategia.
- **Si la respuesta es SÍ:** Reflexiona si ese proceso está **automatizado** para ejecutarse en cada cambio de código y detectar regresiones temprano.

6: Plan de Pruebas con Enfoque en Calidad y Seguridad en Dockers



Plan de Pruebas con Enfoque en Calidad y Seguridad con Docker

Importancia de un “Plan Robusto”

Un plan de pruebas sólido es fundamental para asegurar la calidad y seguridad en entornos modernos de desarrollo. La estructura del plan debe contemplar validaciones exhaustivas que garanticen la integridad de cada componente del sistema.

Contexto del Caso Semestral

Nuestro enfoque utiliza imágenes Docker como base fundamental para las pruebas, aprovechando su capacidad de crear entornos reproducibles y consistentes que permiten validaciones precisas y controladas.

Objetivos Principales

Garantizar la integridad, confiabilidad y protección en entornos de contenedores mediante estrategias sistemáticas de validación, verificación y monitoreo continuo de seguridad.



¿Qué es una Imagen Docker y por qué es clave en pruebas?

Definición y características

Una imagen Docker es una plantilla inmutable que encapsula todos los componentes necesarios para ejecutar una aplicación: sistema operativo, bibliotecas, dependencias y configuraciones específicas. Esta inmutabilidad garantiza una ejecución consistente en cualquier entorno.

Beneficios clave

- Permite crear contenedores replicables y uniformes, independientemente del entorno de despliegue.
- Facilita un control de versiones preciso, con trazabilidad de cambios y posibilidad de revertir estados anteriores.
- Ofrece personalización total para entornos de prueba específicos o simulaciones controladas.
- Asegura aislamiento entre aplicaciones, evitando conflictos por dependencias compartidas.



Levantamiento de Imágenes Docker en el Caso Semestral

El proceso de levantamiento de imágenes Docker requiere una metodología estructurada que garantice la reproducibilidad y calidad de los entornos de prueba. Este proceso comienza con la definición clara de requisitos y dependencias.

01

Creación y Captura

Proceso sistemático de creación de imágenes específicas para pruebas, documentando cada paso y decisión técnica tomada durante el proceso de construcción.

02

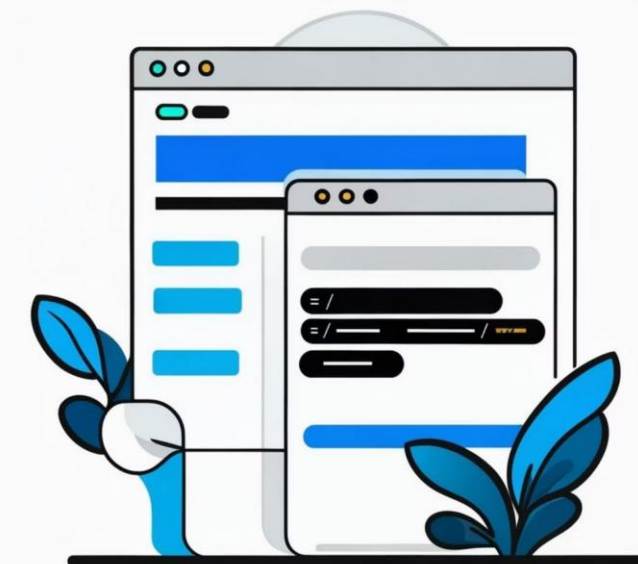
Definición con Dockerfile

Uso de Dockerfile para definir entornos exactos con configuraciones precisas, garantizando que cada dependencia y configuración esté explícitamente declarada.

03

Implementación Práctica

Ejemplo: Dockerfile personalizado para aplicación Node.js con versiones específicas, variables de entorno controladas y optimizaciones para pruebas.



Del Dockerfile al Contenedor en Ejecución



Este diagrama ilustra el flujo completo desde la definición en Dockerfile hasta la ejecución del contenedor, mostrando cómo cada etapa contribuye a la creación de un entorno de pruebas robusto y reproducible.

Estrategias para el Diseño del Plan de Pruebas con Imágenes Docker

Objetivos Claros de Calidad y Seguridad

Establecer métricas medibles y criterios de aceptación específicos que definan el éxito de las pruebas. Estos objetivos deben alinearse con los estándares de la industria y requisitos del negocio.

Selección de Imágenes Base

Priorizar imágenes oficiales y minimizadas como Alpine Linux, que ofrecen menor superficie de ataque y mayor eficiencia. La elección correcta de la imagen base impacta directamente en seguridad y rendimiento.

Versionado y Etiquetado

Implementar sistema robusto de versionado semántico y etiquetado descriptivo para garantizar trazabilidad completa. Cada versión debe estar documentada con cambios específicos y fecha de creación.

Automatización Integral

Automatizar el levantamiento, construcción y despliegue de imágenes para pruebas mediante pipelines CI/CD que reduzcan errores humanos y aceleren el proceso de validación.

Buenas Prácticas de Seguridad en Imágenes Docker



Imágenes Endurecidas

Utilizar imágenes Hardened específicamente diseñadas para minimizar vulnerabilidades conocidas y proporcionar una base segura desde el inicio del desarrollo.



Minimización de Capas

Reducir capas y dependencias innecesarias para disminuir la superficie de ataque. Cada capa adicional representa un potencial vector de vulnerabilidad.



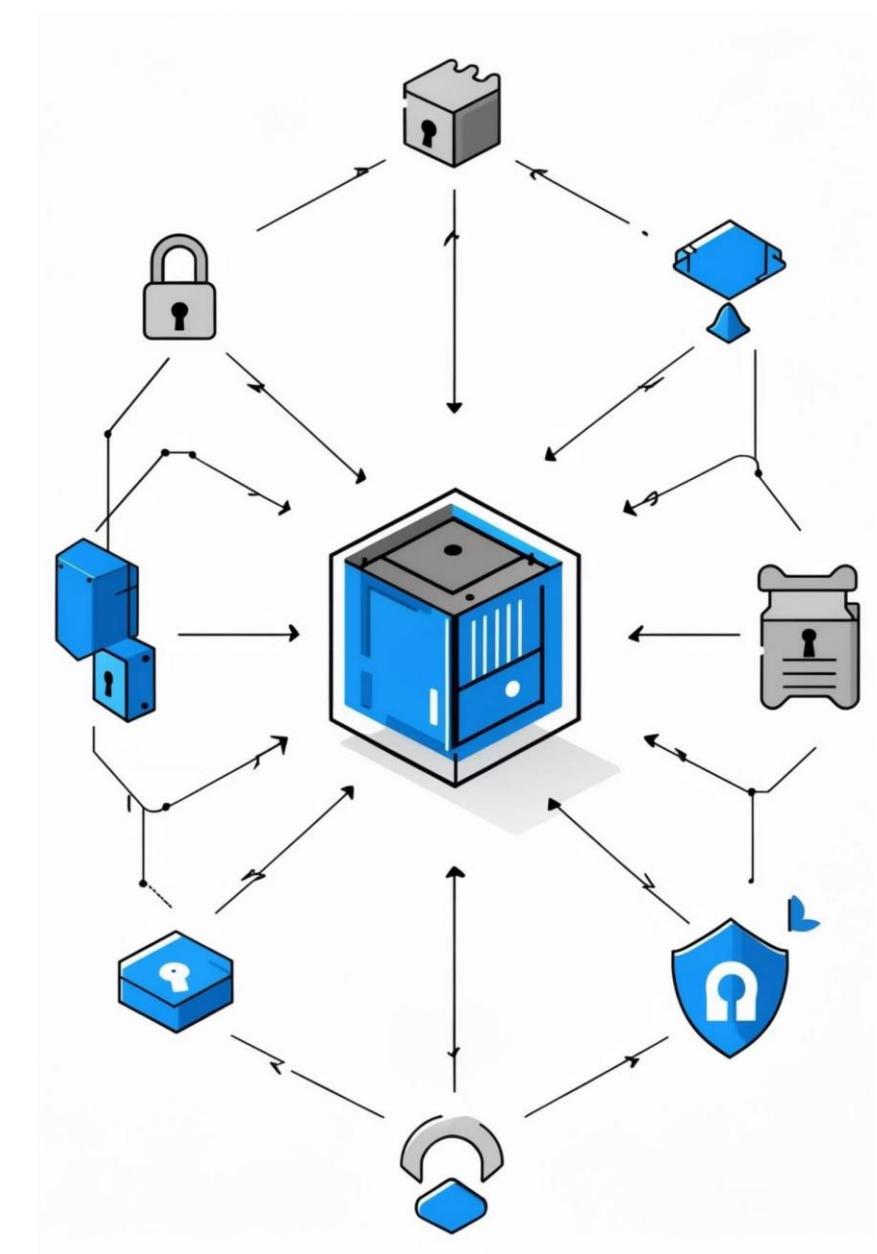
Privilegios Mínimos

Evitar ejecución con privilegios root dentro del contenedor. Implementar usuarios con permisos estrictamente necesarios para las operaciones requeridas.



Aislamiento Avanzado

Implementar espacios de nombres Linux y restricciones de capacidades para crear barreras de seguridad adicionales entre contenedores y el sistema host.

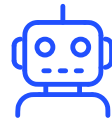


Herramientas recomendadas para planificar pruebas



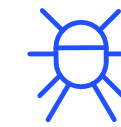
Gestión de casos

TestRail y Zephyr para organizar, ejecutar y rastrear casos de prueba de manera centralizada y colaborativa.



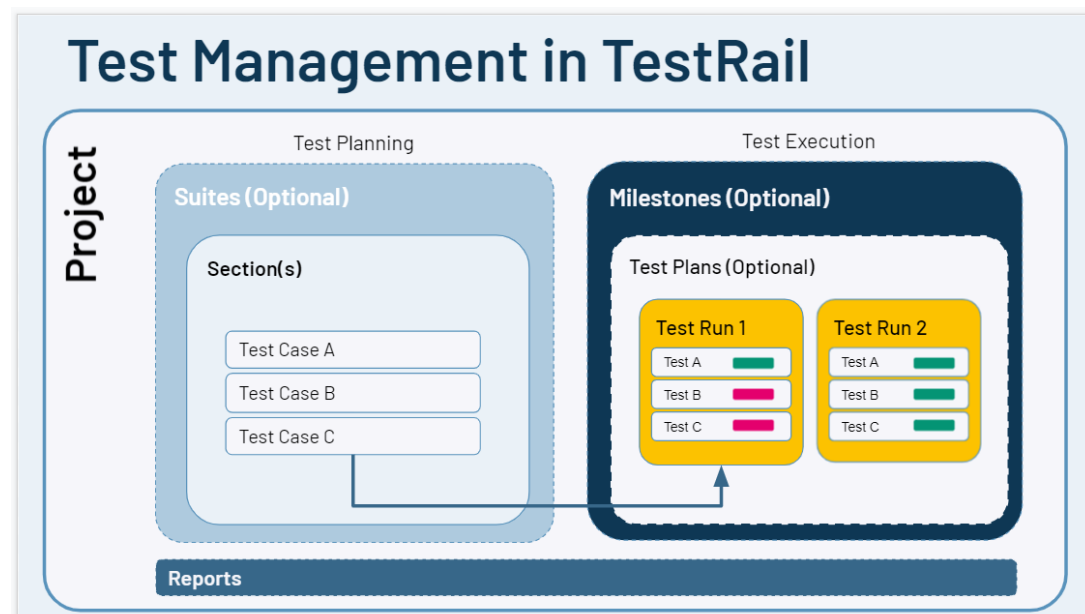
Automatización

Selenium y Appium permiten automatizar pruebas web y móviles, reduciendo esfuerzo manual repetitivo.



Seguimiento de incidencias

Jira y Bugzilla facilitan el registro, seguimiento y resolución de defectos con trazabilidad completa.



Conclusión: Garantiza Calidad y Seguridad desde el Diseño del Plan de Pruebas

Base del Éxito

Un plan de pruebas bien diseñado es el fundamento para desarrollar software confiable, seguro y de alta calidad.

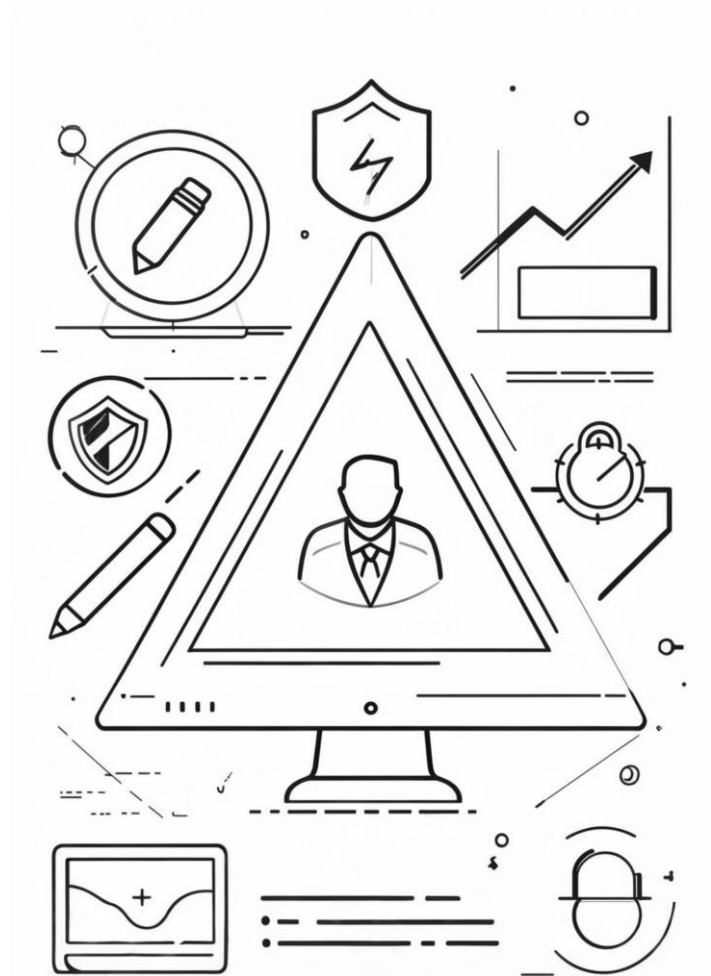
Integración Temprana

Incorporar calidad y seguridad desde el inicio reduce significativamente costos y riesgos a largo plazo.

Mejora Continua

Invertir en planificación estructurada, colaboración efectiva y aprendizaje constante garantiza el éxito sostenible.

"La calidad nunca es un accidente; siempre es el resultado de un esfuerzo inteligente, una planificación cuidadosa y una ejecución disciplinada."



¡Comienza hoy! Aplica estos principios y estrategias para crear planes de pruebas que transformen la calidad y seguridad de tus proyectos de software.

Cierre: Hacia la Excelencia en el Testing

Un plan de pruebas efectivo es un marco estratégico que guía todas las actividades de validación del software⁹. Al integrar calidad y seguridad desde el diseño, no solo reduces riesgos, sino que garantizas entregas confiables al cliente.

- **Puntos clave para recordar:**
 - Implementar **Shift-Left Testing** para detectar fallos en etapas tempranas.
 - Asegurar la **independencia del equipo de testing** para mantener la objetividad.
 - Utilizar **métricas e indicadores** para medir la eficacia del proceso.

¡Has finalizado con éxito este recorrido por las estrategias de diseño!

Bibliografía y Referencias Técnicas

Marcos de Trabajo y Estándares Referenciados

El documento basa sus estrategias en los siguientes estándares internacionales y marcos de buenas prácticas:

- **IEEE 829:** Estándar utilizado para la creación de plantillas estructuradas de planes de pruebas.
- **OWASP (Open Web Application Security Project):**
 - **WSTG (Web Security Testing Guide):** Guía aplicada para el diseño de casos de prueba de seguridad y validación contra inyecciones SQL.
 - **OWASP Top 10:** Referencia para la identificación de las vulnerabilidades más críticas en aplicaciones web.
- **ISTQB (International Software Testing Qualifications Board):** Syllabus y mejores prácticas para la metodología de pruebas y principios fundamentales del testing.
- **NIST Cybersecurity Framework:** Marco de trabajo para la gestión de riesgos de ciberseguridad.
- **Metodologías Ágiles:** Integración con ciclos de desarrollo ágil, DevOps y tuberías de CI/CD (Integración Continua y Despliegue Continuo).

Herramientas de Software Mencionadas

Para la ejecución de las estrategias descritas, se referencian las siguientes tecnologías:

- **Automatización:** Selenium, Cypress, Playwright y Appium.
- **Seguridad:** OWASP ZAP, Burp Suite y SonarQube.
- **Gestión y Seguimiento:** Jira, TestRail, Azure DevOps, Zephyr y Bugzilla.
- **Infraestructura:** Docker (uso de imágenes inmutables y contenedores para entornos reproducibles).

Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación